

Evaluating State Tracking in LLM-based Interactive Narrative Generation

Pujui Tung

lsf5322@gmail.com

Abstract

Large Language Models (LLMs) often struggle to maintain narrative coherence and rule adherence over extended multi-turn interactions, partly due to difficulties in tracking and updating dynamic world states. This paper investigates whether integrating an external “Observer LLM” can improve state consistency in dialogue-based game simulations compared with standalone LLM architectures. We evaluate four distinct story simulation frameworks: (1) a standalone LLM functioning as the sole Game Master with implicit state tracking, where numerical state values are hidden from the output, (2) a standalone LLM with explicit state tracking, where numerical changes and consumed rules are revealed after each turn, (3) a deterministic programmatic state tracker, and (4) a collaborative framework utilizing an auxiliary Observer LLM to monitor the Game Master and maintain an external representation of the game state. Experimental results demonstrate that relying solely on a standalone LLM for implicit state maintenance is unreliable, yielding an average state-tracking accuracy of only 0.10 in the corresponding experiments. In contrast, integrating an Observer LLM substantially improves state-tracking performance, achieving an average accuracy of approximately 0.77 in the extended 10-turn setting. Furthermore, the Observer LLM maintains relatively stable state-tracking performance in longer interactions, suggesting that external observation enables more efficient knowledge extraction from conversations. These findings demonstrate the potential of external observational mechanisms for improving state consistency in LLM-based interactive narrative generation.

1 Introduction

Large language models (LLMs) have demonstrated strong capabilities in generating fluent and coherent narratives and supporting interactive dialogue.

However, maintaining consistent representations of entities and world states across multiple interactions remains challenging (Papalampidi et al., 2022).

As a narrative progresses, previously established information and world states should remain consistent with subsequent descriptions and events. Failures to maintain such information can result in contradictions, incorrect consequences, and inconsistencies in the evolving narrative (Ghosh et al., 2023).

This issue is particularly important in interactive narrative systems, where the generated response should remain consistent with the world state established by previous interactions. Unlike conventional text generation, an interactive Game Master must not only produce plausible narrative responses but also account for changes caused by player actions and previously established rules. For example, when an action consumes a limited resource, subsequent responses should reflect the updated resource state. Failure to track such changes can lead to contradictory outcomes and undermine the consistency of the interaction.

External memory, retrieval mechanisms, and structured representations such as knowledge graphs have been explored to provide LLMs with information beyond their immediate conversational context (Lewis et al., 2021; Gao et al., 2024; Gutiérrez et al., 2025). Such mechanisms can help preserve and retrieve relevant information that may otherwise be difficult for the model to maintain during generation.

However, in interactive narrative, the relevant information is not static: player actions continuously modify the underlying world state. Previous work on textual world models and generative agents has explored mechanisms for representing, retrieving, and maintaining information about evolving environments (Ammanabrolu and Riedl, 2019; Park et al., 2023). An LLM-based Game Master must

therefore not only retrieve or remember previously established information, but also update the world state according to player actions and apply these changes consistently in subsequent interactions. The reliability of this dynamic state tracking remains an open empirical question.

To investigate this issue, we developed a prototype LLM-based Game Master within a customized interactive narrative environment. The simulation scenario involves a spaceship crash, where players must survive in an abandoned space mining station and successfully send out a distress signal before resources run out.

The environment maintains explicit world-state variables, including resource levels and event conditions, while the Game Master is responsible for interpreting player actions and updating the narrative accordingly. Rather than proposing a complete solution to state-consistency problems, this study uses the environment as a controlled setting to examine how reliably an LLM can track and maintain dynamic world states across multiple interactions.

Research Question: *To what extent can an LLM-based Game Master maintain consistent world states and follow dynamically changing game rules across multiple interactions?*

2 Related Work

2.1 Interactive Narratives and Game Master Challenges

In domains characterized by complete information and explicit rules, such as chess and Go, artificial intelligence has already matched or surpassed human proficiency. However, open-ended environments driven by natural language, epitomized by tabletop role-playing games like Dungeons & Dragons (D&D), present the next frontier for AI (Louis and Sutton, 2018).

Evaluating AI systems within such highly open-ended settings—where dialogue must simultaneously reference background knowledge and adhere to complex rules—serves as a robust measure of linguistic and reasoning capabilities. Previous empirical analyses by Callison-Burch et al. (2022) revealed that when a Game Master LLM is forced to simultaneously manage narrative generation and rigorous state tracking, performance often degrades. This underscores the necessity of externalized support mechanisms to assist narrative models in preserving logical coherence and engaging plots.

2.2 Narrative Consistency and State Tracking

Recent studies have also investigated consistency in interactive narratives, including whether agents preserve previously established narrative commitments during interaction. Ma et al. (2026) introduced the concept of narrative commitment preservation to evaluate whether generated events remain consistent with previously established constraints. These studies highlight the importance of evaluating consistency not only at the level of individual responses, but also across the evolving state of an interaction.

Text-adventure games provide a particularly useful setting for this type of evaluation because they combine structured states, explicit rules, and sequential interactions. Such environments make it possible to evaluate whether an agent’s actions and responses remain consistent with the evolving game state. Recent work such as Hu et al. (2026) further demonstrates the use of game environments for systematically evaluating LLM behavior in interactive settings. These studies motivate the use of controlled game environments for examining state tracking and rule adherence in interactive narrative systems.

2.3 External Memory and Knowledge Graph Integration

To mitigate these limitations, recent paradigms have increasingly incorporated external architectures to augment LLM decision-making. Park et al. (2023) introduced "Generative Agents" (the "Sandbox Village" framework), which simulate a memory stream consisting of observation, retrieval, and reflection mechanisms to provide external references for agent behavior.

Similarly, Ammanabrolu and Riedl (2019) proposed integrating dynamic knowledge graphs into text-adventure games, enabling narrative LLMs to more effectively select rational actions, continuously update plot contexts, and enhance the overall interpretability of the generated outputs. Building upon these insights, our work focuses on evaluating state tracking under controlled resource dynamics and dynamically introduced rules.

Existing studies have demonstrated the potential of external memory and structured knowledge for interactive narrative generation. However, our study focuses specifically on isolating and evaluating the state-tracking component under controlled resource dynamics and dynamically introduced

rules.

3 Methodology

3.1 Simulation Environment and Narrative Setting

To evaluate narrative generation and state tracking, we implement a custom narrative simulation platform powered by gpt-4o-mini (OpenAI et al., 2024) acting as the central Game Master LLM. The simulation scenario places the player in a survival crisis: following a spacecraft accident, the player must evacuate to an abandoned space mining station and strategically manage their actions under severe resource scarcity.

The simulation environment is governed by three core state attributes: power, oxygen, and stress. The game terminates under two failure conditions: resource depletion (i.e., when either power or oxygen drops to zero, indicating a total failure of life support systems) or psychological collapse (i.e., when stress exceeds a threshold of 10 points). To survive, the player must issue conversational actions to explore the environment, perform repairs, and ultimately transmit a distress signal.

3.2 Initial States and Predefined Consumption Rules

Inspired by resource-constrained evaluation frameworks in strategic environments (Yuan et al., 2025), we establish a controlled environment governed by three core states and a foundational set of consumption rules. Furthermore, we anticipate incorporating a Player LLM in future work to establish a fully automated game evaluation mechanism.

At the beginning of the simulation, the world state is initialized with 40 power, 20 oxygen, and 0 stress. Each player dialogue input represents one discrete simulation turn. State transitions are governed by a set of predefined deterministic consumption rules as follows:

1. **Power:** Each turn incurs a baseline cost of (-2); specific actions induce additional costs, such as area illumination (-1), operating a plasma welder (-10), and transmitting a distress signal (-5).
2. **Oxygen:** Each turn incurs a baseline cost of (-1); exploratory actions induce an additional cost of (-2).

3. **Stress:** Each turn incurs a baseline accumulation of (+1); actions performed in dark environments incur an additional (+2).

4 Experiments and Evaluation

4.1 Experimental Settings

To comprehensively evaluate state-tracking efficacy, we conduct four experimental settings with different interaction horizons and rule conditions.

1. **Constrained Setting:** The Game Master LLM is strictly prohibited from introducing consumption rules outside the predefined rulebook. The simulation is driven by a fixed sequence of 7 scripted player action turns.
2. **Open-Ended Setting:** The Game Master LLM is permitted to dynamically introduce new consumption rules not present in the predefined rulebook, provided that the impact of these rules is explicitly defined. These newly introduced rules are manually verified to determine whether they are properly factored into state tracking. This setting is driven by a fixed sequence of 8 scripted player action turns. According to manual trace calculations under static rules, the simulation should terminate at the 8th turn due to stress exceeding the maximum threshold. However, due to logging discrepancies, we unfortunately observed only a single instance of game termination independent of state values. This discrepancy prevented reliable evaluation of termination correctness and motivated us to focus the current analysis on per-turn state tracking rather than terminal-state prediction.
3. **Extended Open-Ended Setting (10 Turns):** To further examine framework robustness under a longer interaction horizon, we extend the open-ended setting to a fixed sequence of 10 scripted player action turns, executed across 5 independent trials for all four frameworks.
4. **Long-Horizon Setting (20 Turns):** To isolate the effect of interaction length on the two best-performing architectures, we further evaluate the Standalone LLM (Explicit) and Observer LLM frameworks under a fixed sequence of 20 scripted player action turns, executed across 10 independent trials. Note that the player action sequences used in the 10-turn and 20-turn settings are not identical, and

this discrepancy is discussed as a limitation in Section 5.3.

4.2 Evaluated Frameworks

We evaluate four framework configurations that differ in how world-state updates are maintained and represented. Table 1 summarizes the key architectural differences among these configurations. Figure 1 illustrates the overall information flow between the Player, Game Master LLM, state-tracking components, and persistent world state.

The first two configurations delegate state maintenance to the Game Master LLM, while the third and fourth configurations externalize state updates to dedicated components.

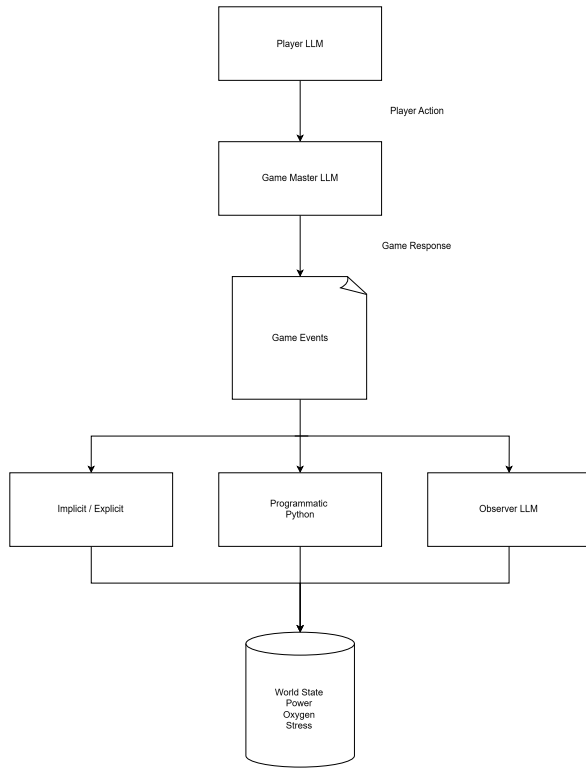


Figure 1: Information flow between the Player, Game Master LLM, state-tracking components, and persistent world state across the evaluated frameworks.

1. **Standalone LLM (Implicit):** The Game Master LLM is tasked with both narrative generation and implicit state tracking. Numerical changes are withheld until the game terminates. The required output format is structured as: [Environment][New Rules].
2. **Standalone LLM (Explicit):** Similar to the first framework, the Game Master LLM handles narrative generation, but performs explicit state tracking. The current state and

the specific rules applied are displayed at the end of each turn. The required output format is: [Environment][Current State][Applied Rules][New Rules].

3. **Deterministic Programmatic State Tracker:** The Game Master LLM focuses solely on narrative generation, while state calculations are handled deterministically via Python code based on per-turn actions and fed back to the LLM to assist in subsequent story generation. The input/output formats are structured as follows: Player input comprises [Observer’s LastState Resolution][Player Action], while Game Master output consists of [Environment][New Rules].
4. **Collaborative Framework with an Auxiliary Observer LLM:** Following the Game Master’s narrative generation, the dialogue history between the player and the Game Master is passed to an auxiliary Observer LLM. The Observer computes the state updates and feeds them back to the Game Master to guide narrative progression. The formatting is structured as follows: Player input comprises [Observer’s LastState Resolution][Player Action], Game Master output consists of [Environment][New Rules], and Observer output is formatted as [State][Applied Rules][New Rules].

4.3 Results and Analysis

State Tracking Accuracy was evaluated across independent trials for each framework, with five trials used for the short- and extended-horizon settings and ten trials used for the 20-turn long-horizon setting. State Tracking Accuracy measures the proportion of evaluated state updates that exactly match the ground-truth state after each turn. For the open-ended setting, we additionally evaluate Dynamic Rule Integration, which measures the proportion of newly introduced state-affecting rules that are correctly incorporated into subsequent state updates. Figure 2, Figure 3 and Figure 4 summarize state-tracking performance across the experimental settings, while Table 2 reports dynamic rule integration results, while Table 2 reports dynamic rule integration results.

4.3.1 Constrained Setting Performance

In the constrained setting, the Observer LLM achieves the highest overall accuracy (0.58), followed by the programmatic tracker (0.54), while

Table 1: Framework Comparison

Framework	State Update	State Representation	External Component
Implicit LLM	Game Master LLM	Hidden	None
Explicit LLM	Game Master LLM	Explicit	None
Programmatic	Python	External	Python Tracker
Observer LLM	Observer LLM	External	Observer LLM

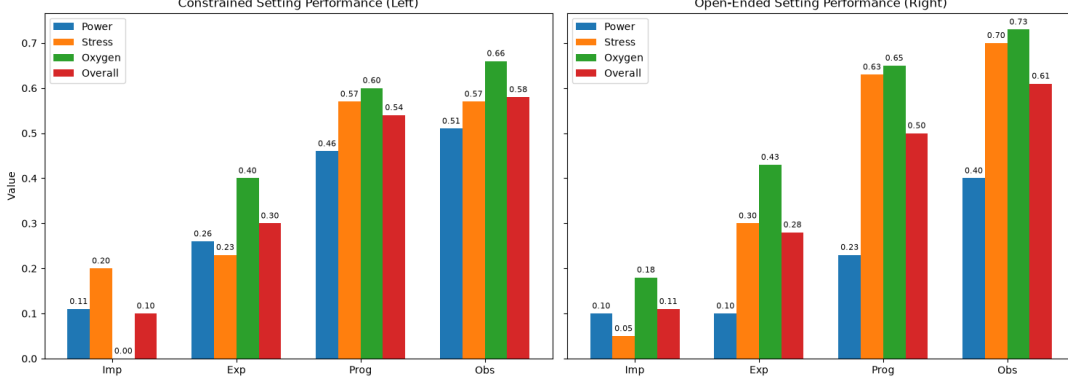


Figure 2: State-tracking accuracy across the four frameworks. (left) Constrained setting without dynamic rule creation. (right) Open-ended setting with dynamically introduced rules.

the implicit and explicit LLM configurations achieve lower overall accuracies of 0.10 and 0.30, respectively.

Analysis: These results demonstrate that within this simulation platform, burdening a single LLM with both narrative generation and state maintenance fails to compute states accurately. Specifically, implicit state tracking yields near-zero reliability, particularly in maintaining oxygen metrics (0.00). While Framework (3) manages per-turn baseline consumption through deterministic code, it remains limited. In contrast, the auxiliary Observer LLM (Framework 4) successfully tracks state progression by reasoning over the dialogue history, achieving the highest baseline accuracy (0.58).

4.3.2 Open-Ended Setting Performance

For the open-ended setting, we additionally evaluate Dynamic Rule Integration, which measures the proportion of newly introduced state-affecting rules that are correctly incorporated into subsequent state updates.

Table 2 further summarizes the number of dynamically generated rules and the number of state-affecting rules that were correctly incorporated into subsequent state updates.

The Observer LLM correctly integrated all six

state-affecting rules, whereas the implicit, explicit, and programmatic frameworks integrated six of eight, one of eight, and four of fifteen state-affecting rules, respectively.

Analysis: Once the Game Master LLM is granted the freedom to dynamically introduce new rules, the Deterministic Programmatic State Tracker (Framework 3) struggles to incorporate these unprogrammed constraints, causing a sharp drop in Power tracking accuracy (plummeting from 0.46 down to 0.23). Interestingly, although Standalone LLM Explicit (Framework 2) outperforms its implicit counterpart in general state logging, it similarly fails to apply newly created rules to active state updates. In contrast, the Auxiliary Observer LLM (Framework 4) demonstrates superior adaptability and robustness; despite the introduction of dynamic rules, it maintains the highest overall accuracy (0.61) and successfully integrates 100% of the effective new rules (6 out of 6) into state updates. In our experimental setting, the Observer LLM achieved the highest state-tracking accuracy, particularly when dynamically introduced rules were present.

Table 2: Dynamic Rule Integration

Open-Ended Setting	Correctly Integrated	State-affecting	Total New Rules
Implicit LLM	6	8	11
Explicit LLM	1	8	8
Programmatic	4	15	15
Observer LLM	6	6	6
Extended Setting	Correctly Integrated	State-affecting	Total New Rules
Implicit LLM	0	2	4
Explicit LLM	4	4	4
Programmatic	4	6	6
Observer LLM	2	2	2
Long-Horizon Setting	Correctly Integrated	State-affecting	Total New Rules
Explicit LLM	5	5	5
Observer LLM	0	1	4

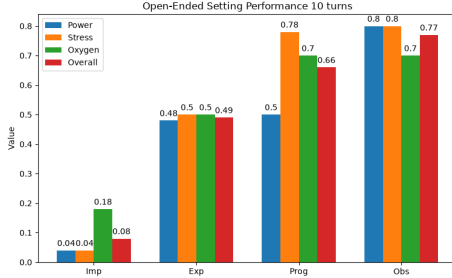


Figure 3: Extended Open-Ended Setting (10 Turns)

4.3.3 Extended Setting Performance (10 Turns)

Figure 3 illustrates the performance of each architecture when expanded to 10 rounds of action.

Analysis: Consistent with the constrained and open-ended results, we found that the Explicit and Programmatic frameworks achieve near-perfect integration of state-affecting rules when only a single rule applies per turn. However, when two rules overlap within the same turn, these two architectures tend to apply only one of them while disregarding the other, suggesting these frameworks fail specifically when multiple concurrent rules must be composed within a single update, rather than failing at rule detection itself.

The Observer framework continues to lead in overall accuracy (0.77) and, unlike the other three frameworks, shows evidence of correctly composing two or more concurrent rule effects within a single turn.

4.3.4 Long-Horizon Setting Performance (20 Turns)

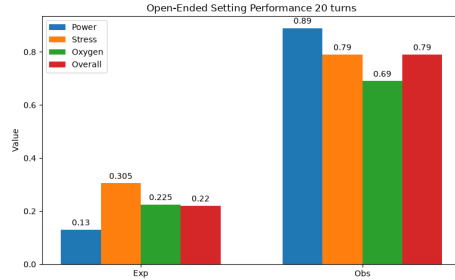


Figure 4: Long-Horizon Open-Ended Setting (20 Turns)

To compare the two frameworks under extended dialogue, we selected the Explicit LLM and Observer LLM for a 20-round evaluation. The player actions for the first 10 rounds were revised from the 10-turn experiment to correct earlier scripting inconsistencies, while the actions for the final 10 rounds reused the first 10 with minor edits to keep newly introduced rules consistent with the predefined rulebook.

Figure 4 illustrates the performance of each architecture when expanded to 20 rounds of action.

Analysis: The differences between the two frameworks become apparent as the number of interactions increases and the rules used become more complex. The Explicit LLM’s overall accuracy collapses from 0.49 (10 turns) to 0.22, suggesting that cumulative baseline arithmetic — rather than rule detection — is the primary failure point over long horizons; notably, its rule-integration rate remains perfect (5/5), reinforcing the earlier

observation that single-rule application is not the bottleneck, but rather the compounding of per-turn baseline costs across many turns.

The Observer framework maintains a comparatively high overall accuracy (0.79), but its rule-integration performance drops to 0/1 state-affecting rules, the first instance in this study where the Observer framework fails to correctly incorporate a dynamically introduced rule.

Given that only one state-affecting rule was introduced during these 10 trials, this result should be treated as preliminary; however, it suggests that the Observer framework’s advantage in composing concurrent or novel rule effects may itself begin to erode at longer horizons, aligning with the cumulative-drift concern motivating this line of investigation.

5 Conclusion & Future Work

5.1 Conclusion

In this paper, we presented a dialogue-based simulation testbed designed to evaluate the state-tracking performance of four distinct frameworks across experimental settings of varying lengths. Although each experiment was conducted with a limited number of independent trials, the preliminary results indicate that the simulation platform can reveal substantial differences in state-tracking behavior across architectural configurations. Specifically, our findings suggest that requiring a standalone LLM to simultaneously perform narrative generation and state maintenance is unreliable. Furthermore, the Observer LLM demonstrated greater adaptability to dynamically introduced rules than the deterministic tracker in our current experimental setting.

Beyond quantitative metrics, our qualitative observations revealed intriguing failure modes in LLM-based narrative generation. For instance, in one trial set within an abandoned space mining station, the Game Master LLM generated a severe physical and logical contradiction by describing the player as "opening a window, allowing outside oxygen to flood the station and secure a successful escape." This failure illustrates that state inconsistency is not limited to numerical bookkeeping; it can also manifest as violations of physical and causal constraints.

5.2 Future Work

Building upon these insights, we intend to pursue several directions for future research:

1. **Diverse Narrative Backgrounds:** Expanding the simulation testbed to different thematic scenarios to test framework generalizability.
2. **Dynamic Knowledge Graphs:** Integrating structured knowledge graphs to enhance the system’s external memory and long-term context retention.
3. **Automated Evaluation via Player LLMs:** Replacing scripted inputs with a Player LLM to establish a fully automated, end-to-end evaluation loop.
4. **Further Long-Horizon Scaling and Robustness Evaluation:** Extending the evaluation beyond 20 turns and testing whether the observed state-tracking robustness of the Observer framework persists under substantially longer interactions and more frequent dynamic rule introduction.

5.3 Limitations

The 10-turn and 20-turn extended experiments used player action sequences that were not held identical across settings, which introduces a confound: differences in accuracy across turn lengths may partly reflect differences in scripted content rather than interaction length alone. Additionally, the 20-turn setting was only evaluated for the Explicit and Observer frameworks, precluding a full four-way comparison at this horizon. Future work should standardize action sequences across turn lengths and extend the Implicit and Programmatic frameworks to the 20-turn setting for a complete comparison.

References

- Prithviraj Ammanabrolu and Mark O. Riedl. 2019. [Playing text-adventure games with graph-based deep reinforcement learning](#). *Preprint*, arXiv:1812.01628.
- Chris Callison-Burch, Gaurav Singh Tomar, Lara J. Martin, Daphne Ippolito, Suma Bailis, and David Reitter. 2022. [Dungeons and dragons as a dialog challenge for artificial intelligence](#). In *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing*, page 9379–9393. Association for Computational Linguistics.

- Yunfan Gao, Yun Xiong, Xinyu Gao, Kangxiang Jia, Jinliu Pan, Yuxi Bi, Yi Dai, Jiawei Sun, Meng Wang, and Haofen Wang. 2024. [Retrieval-augmented generation for large language models: A survey](#). *Preprint*, arXiv:2312.10997.
- Sayontan Ghosh, Mahnaz Koupaei, Isabella Chen, Francis Ferraro, Nathanael Chambers, and Niranjan Balasubramanian. 2023. [Pasta: A dataset for modeling participant states in narratives](#). *Preprint*, arXiv:2208.00329.
- Bernal Jiménez Gutiérrez, Yiheng Shu, Yu Gu, Michihiro Yasunaga, and Yu Su. 2025. [Hipporag: Neurobiologically inspired long-term memory for large language models](#). *Preprint*, arXiv:2405.14831.
- Lanxiang Hu, Mingjia Huo, Yuxuan Zhang, Haoyang Yu, Eric P Xing, Ion Stoica, Tajana Rosing, Haojian Jin, and Hao Zhang. 2026. [Imgame-bench: How good are llms at playing games?](#) In *International Conference on Learning Representations*, volume 2026, pages 81308–81356.
- Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. 2021. [Retrieval-augmented generation for knowledge-intensive nlp tasks](#). *Preprint*, arXiv:2005.11401.
- Annie Louis and Charles Sutton. 2018. [Deep dungeons and dragons: Learning character-action interactions from role-playing game transcripts](#). In *Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 2 (Short Papers)*, pages 708–713, New Orleans, Louisiana. Association for Computational Linguistics.
- Yingpeng Ma, Jianhao Yan, Bei Shi, Ka Hou Kam, Runnan Wang, Xuebo Liu, Yulong Chen, Yue Zhang, and Derek F. Wong. 2026. [Can llm agents stick to the script? a benchmark for long-horizon consistency in interactive narratives](#). *Preprint*, arXiv:2608.08160.
- OpenAI, :, Aaron Hurst, Adam Lerer, Adam P. Goucher, Adam Perelman, Aditya Ramesh, Aidan Clark, AJ Ostrow, Akila Welihinda, Alan Hayes, Alec Radford, Aleksander Mądry, Alex Baker-Whitcomb, Alex Beutel, Alex Borzunov, Alex Carney, Alex Chow, Alex Kirillov, and 401 others. 2024. [Gpt-4o system card](#). *Preprint*, arXiv:2410.21276.
- Pinelopi Papalampidi, Kris Cao, and Tomas Kocisky. 2022. [Towards coherent and consistent use of entities in narrative generation](#). *Preprint*, arXiv:2202.01709.
- Joon Sung Park, Joseph C. O’Brien, Carrie J. Cai, Meredith Ringel Morris, Percy Liang, and Michael S. Bernstein. 2023. [Generative agents: Interactive simulacra of human behavior](#). *Preprint*, arXiv:2304.03442.
- Xiaopeng Yuan, Xingjian Zhang, Ke Xu, Yifan Xu, Lijun Yu, Jindong Wang, Yushun Dong, and Haohan Wang. 2025. [Tracing llm reasoning processes with strategic games: A framework for planning, revision, and resource-constrained decision making](#). *Preprint*, arXiv:2506.12012.